Linux & Shell Scripting

Contest Preparation Notes

Comprehensive Reference

1. Linux Architecture & Core Concepts

The Kernel

The Linux kernel is the core of the OS. It sits between hardware and user applications, handling:

- **CPU scheduling** — decides which process runs and when
- **Memory management** — allocates/deallocates RAM, manages virtual memory
- **Device drivers** — communicates with hardware through a standardised interface
- **System calls** — the API between user programs and the kernel (open(), read(), fork(), etc.)

> **Note:** *Users never interact with the kernel directly. All commands go through a shell.*

The Shell

A shell is a command interpreter. Common shells:

Shell	Notes
`bash`	Bourne Again Shell — default on most Linux distros
`zsh`	Extended features, popular for interactive use
`sh`	POSIX-compliant, minimal — used for portable scripts
`fish`	User-friendly but non-POSIX

> **Note:** *Your shell is a user-space program. Shell scripts make system calls to the kernel for low-level operations.*

Process vs. Shell Builtins

Every command spawns a **child process** of the shell. But some commands — called **shell builtins** — run inside the shell process itself (e.g. `cd`, `echo`, `export`). Builtins can affect the shell's own state; external commands cannot.

2. Directory Structure Deep Dive

Everything in Linux lives under root `/`. This is the Filesystem Hierarchy Standard (FHS).

Directory	Purpose
/bin	Essential user binaries (ls, cp, mv, grep)
/sbin	System binaries — root-only tools (ifconfig, fsck)
/etc	Configuration files — text-based, human-readable
/var/log	System and application logs
/home	User home directories
/root	Root user's home — NOT under /home
/tmp	Temporary files — wiped on reboot
/usr/bin	Non-essential user binaries
/proc	Virtual filesystem — live kernel/process info (not on disk)
/dev	Device files (/dev/sda, /dev/null, /dev/random)
/lib	Essential shared libraries

Important Special Files

Path	What it is
/dev/null	Discards everything written; returns EOF when read
/dev/zero	Returns infinite null bytes — used to wipe disks
/dev/random	Cryptographically secure random bytes
/etc/passwd	User account database (no actual passwords since shadow)
/etc/shadow	Hashed passwords — root-only readable (permissions: 000)
/etc/crontab	System-wide cron schedule

3. File Permissions & Ownership

Reading ls -l Output

```
drwxr-xr-x  2  samarth  developers  4096  Dec 24 10:00  mydir
|└─┬─┘└─┬─┘  |  └──┬──┘  └────┬─────┘
|  |    |    |     |          └─ owner      group
|  |    |    |     └─ hard link count
|  |    |    └─ others permissions (r-x)
|  |    └─ group permissions (r-x)
|  └─ owner permissions (rwx)
└─ file type: d=directory, -=file, l=symlink
```

Permission Bits

Symbol	Octal	Meaning (file)	Meaning (directory)
r	4	Read file contents	List directory contents
w	2	Modify file	Create/delete files inside
x	1	Execute as program	Enter the directory (cd)
-	0	No permission	No permission

Calculating Octal Permissions

```
7 = 4+2+1 = rwx  (full access)
6 = 4+2+0 = rw-  (read and write)
5 = 4+0+1 = r-x  (read and execute)
4 = 4+0+0 = r--  (read only)
0 = 0+0+0 = ---  (no access)
```

Common Permission Patterns

chmod	Pattern	Typical Use
755	rwxr-xr-x	Executable scripts and directories
644	rw-r--r--	Regular files, config files
600	rw-------	Private files (SSH keys, secrets)
777	rwxrwxrwx	World-writable — INSECURE, avoid
700	rwx------	Owner-only executables
000	---------	No access for anyone

chmod Syntax

```
# Numeric (octal)
chmod 755 script.sh

# Symbolic
chmod u+x script.sh       # Add execute for owner
chmod g-w file.txt        # Remove write from group
chmod o=r file.txt        # Set others to read-only
chmod a+x script.sh       # Add execute for all
```

Symbolic Links vs. Hard Links

	Symbolic Link	Hard Link
Created with	ln -s target link	ln target link
Points to	Path (string)	Inode (data on disk)
Works across filesystems	Yes	No
If original deleted	Dangling link — broken	File still accessible
Can link directories	Yes (with care)	No

> **Note:** *A dangling symlink points to a path that no longer exists. The link remains on disk but accessing it gives 'No such file or directory'.*

4. Shell I/O: Streams, Redirection & Pipes

The Three Standard Streams

Stream	FD	Default	Name
stdin	0	Keyboard	Standard Input
stdout	1	Terminal	Standard Output
stderr	2	Terminal	Standard Error

> **Note:** *File descriptors are just integers. When you 'redirect,' you're reassigning where a file descriptor points.*

Redirection Operators

Syntax	Effect
command > file.txt	Redirect stdout to file (overwrites)
command >> file.txt	Redirect stdout to file (appends)
command < file.txt	Feed file as stdin to command
command 2> error.log	Redirect stderr to file
command &> all.log	Redirect both stdout and stderr (bash shorthand)
command > all.log 2>&1	Same — explicit form, works in sh too

Understanding 2>&1

This means: **redirect file descriptor 2 (stderr) to wherever file descriptor 1 (stdout) currently points.**

```
ls /nonexistent > out.log 2>&1     # Both go to out.log

# ORDER MATTERS:
ls /nonexistent 2>&1 > out.log     # WRONG: stderr goes to terminal
                                    # (stdout's ORIGINAL location)
```

The Pipe |

A pipe connects **stdout** of one command to **stdin** of the next. Each command in a pipeline runs in a **subshell**.

```
ps aux | grep python | wc -l
#   │          │           └── counts lines
#   │          └── filters python processes
#   └── lists all processes
```

> **⚠ Warning:** *Pipes only connect stdout to stdin. Stderr flows through to the terminal unless explicitly redirected.*

The tee Command

`tee` reads from stdin and writes to **both a file and stdout simultaneously:**

```
ls -l | tee list.txt          # Displays AND saves to file
ls -l | tee -a list.txt       # Append mode
```

```
ls -l | tee file1 file2        # Write to multiple files
```

Process Substitution

```
diff <(sort file1.txt) <(sort file2.txt)           # Compare sorted versions
while read line; do ...; done < <(cat file.txt)    # Loop runs in current shell
```

> **Note:** *Process substitution < <(...) is the key fix for the pipe-subshell variable scope problem (see Section 9).*

5. The find Command — In Depth

Time-Based Flags

Flag	Meaning	Example
-mtime -N	Modified less than N days ago	-mtime -1 → last 24h
-mtime +N	Modified more than N days ago	-mtime +7 → older than 7 days
-mtime N	Modified exactly N days ago	-mtime 1 → exactly 1 day
-mmin -N	Modified less than N minutes ago	-mmin -60 → last hour
-ctime	Change time (inode change, NOT content)	Permissions, owner changes
-newer file	Modified more recently than file	-newer ref.txt

> ⚠ **Warning:** *-mtime vs -ctime: -mtime tracks when file content was modified. -ctime tracks when the inode changed (includes permission or owner changes). They are NOT interchangeable.*

Size & Type Flags

```
find . -size +100M   # Larger than 100 MB
find . -size -1k     # Smaller than 1 KB
find . -size 0       # Empty files
find . -type f       # Regular files only
find . -type d       # Directories only
find . -type l       # Symbolic links
```

The -exec Flag: \; vs +

```
# Run once PER FILE (safer, slower)
find . -name '*.log' -exec grep 'ERROR' {} \;

# Run ONCE with all files batched (faster)
find . -name '*.log' -exec grep 'ERROR' {} +
```

{} is the placeholder for the found filename. **\;** terminates the -exec command. **+** batches all files into one invocation (like xargs).

-delete and Combining Expressions

```
# Delete all .tmp files older than 7 days
find /tmp -name '*.tmp' -mtime +7 -delete

# AND (default), OR (-o), NOT (-not)
find . -type f -name '*.log' -size +100M          # AND
find . -name '*.log' -o -name '*.txt'             # OR
find . -not -name '*.log'                          # NOT
find . \( -name '*.log' -o -name '*.txt' \) -type f  # Grouping
```

> ⚠ **Warning:** *Always test with -print before adding -delete. There is no undo.*

6. Text Processing Tools

grep — Search for Patterns

```
grep 'ERROR' app.log                # Basic search
grep -r 'config' /etc/              # Recursive
grep -i 'warning' /var/log/syslog   # Case-insensitive
grep -n 'failure' logfile.txt       # Show line numbers
grep -c 'ERROR' app.log             # Count matching LINES
grep -v 'DEBUG' app.log             # Invert — show non-matching lines
grep -A 3 'ERROR' app.log           # 3 lines After match
grep -B 2 'ERROR' app.log           # 2 lines Before match
grep -E 'ERR|WARN' app.log          # Extended regex
```

sed — Stream Editor

```
# Substitution: s/pattern/replacement/flags
sed 's/localhost/prod.db.internal/g' config.yaml    # Print to stdout
sed -i 's/localhost/prod.db.internal/g' config.yaml # Edit in PLACE

# g flag = global (all occurrences per line)
# Without g: only first occurrence per line

sed -n '5,10p' file.txt        # Print only lines 5-10
sed '3d' file.txt              # Delete line 3
sed '/ERROR/d' file.txt        # Delete lines matching pattern
```

> **Note:** *macOS requires: sed -i '' 's/old/new/g' file (empty string after -i). Linux does not need it.*

awk — Field Processing

```
awk '{print $1}' file.txt           # Print first field
awk '{print $NF}' file.txt          # Print last field
awk -F',' '{print $3}' data.csv     # Comma-separated, field 3
awk 'NR==5' file.txt                # Print line 5

# Conditions
awk '$5 == "CRITICAL" {print $3}' file   # Exact match
awk '$5 ~ /CRITICAL/ {print $3}' file    # Regex (contains)
awk '/ERROR/ {print $3}' file            # Pattern on whole line

# Aggregation
awk '{sum += $4} END {print sum}' file
free -m | grep Mem | awk '{print ($4/$2) * 100}'
```

Variable	Meaning
NR	Current line number (Number of Records)
NF	Number of fields on current line
FS	Field separator (default: whitespace)
$0	Entire current line
$1..$N	Field N

> ⚠ **Warning:** *== vs ~: == is exact equality. ~ tests regex containment. $5 ~ /CRITICAL/ is true if field 5 contains CRITICAL anywhere, not just if it equals it exactly.*

sort | uniq — Count and Rank

```
# Classic pattern: count and rank most frequent items
grep 'ERROR' app.log | sort | uniq -c | sort -nr | head -10

sort -n file.txt           # Numeric sort
sort -r file.txt           # Reverse sort
sort -nrk4 file.txt        # Numeric, reverse, by field 4
uniq -c file.txt           # Count occurrences (must sort first!)
uniq -d file.txt           # Show only duplicates
```

7. Process Management

Viewing Processes

```
ps aux              # All processes, BSD format
ps -ef              # All processes, UNIX format
ps -u john          # Processes owned by user john
ps aux | grep nginx  # Filter for specific process
```

Column	Meaning
USER	Process owner
PID	Process ID — unique, assigned at creation
%CPU	CPU usage
%MEM	Memory usage
VSZ	Virtual memory size
RSS	Resident set size — actual RAM used
STAT	R=Running, S=Sleeping, Z=Zombie, T=Stopped
COMMAND	Command that launched the process

Signals

Signal	Number	Default Action	Description
SIGHUP	1	Terminate	Hangup (terminal closed) — nohup intercepts this
SIGINT	2	Terminate	Interrupt (Ctrl+C)
SIGKILL	9	Terminate	Force kill — CANNOT be caught or ignored
SIGTERM	15	Terminate	Graceful termination — default for kill
SIGSTOP	19	Stop	Pause process (Ctrl+Z) — cannot be caught
SIGCONT	18	Continue	Resume a stopped process

```
kill 1234           # Send SIGTERM (15) — graceful
kill -9 1234        # Send SIGKILL — force, no cleanup
kill -SIGHUP 1234   # Send by name
pkill nginx         # Kill by process name
killall nginx       # Kill ALL processes named nginx
```

Job Control

```
command &           # Run in background
Ctrl+Z              # Suspend foreground process
jobs                # List background/suspended jobs
fg %1               # Bring job 1 to foreground
bg %1               # Resume job 1 in background
disown %1           # Detach job — won't be killed when shell exits
```

nohup — Surviving Logout

When you log out, the kernel sends `SIGHUP` to all processes in your session. `nohup` makes a process ignore it:

```
nohup ./long_script.sh &            # Run in background, survive logout
nohup ./long_script.sh > out.txt &  # Specify output (default: nohup.out)
```

> **Note:** *nohup alone doesn't send to background — you still need &. They are complementary: & = background, nohup = survive terminal close.*

8. Bash Scripting Fundamentals

The Shebang

```
#!/bin/bash               # Explicitly use bash
#!/bin/sh                 # POSIX sh — more portable, fewer features
#!/usr/bin/env bash       # Find bash via PATH — recommended
```

Variables & Quoting

```
NAME="DevOps"             # No spaces around =
echo $NAME                # Variable expansion
echo "$NAME"              # Safer: quoted expansion
echo '${NAME}'            # Single quotes: NO expansion, literal
readonly PI=3.14159       # Immutable
unset NAME                # Delete variable
```

Quote type	Behaviour
Single quotes '...'	Everything literal — NO variable or command expansion
Double quotes "..."	Variable ($VAR) and command $(cmd) substitution work
Backticks `cmd`	Command substitution — prefer $(cmd) instead

Special Variables

Variable	Meaning
$0	Script name
$1, $2 ...	Positional arguments
$#	Number of arguments
"$@"	All arguments — individually quoted (ALWAYS use this)
"$*"	All arguments — joined into one string when quoted
$?	Exit code of last command (0 = success)
$$	PID of current shell
$!	PID of last background process

> ⚠ **Warning:** *"$@" vs "$*": Only differs when quoted. "$@" preserves each argument separately ("arg one" "arg two"). "$*" joins them ("arg one arg two"). Always use "$@" when passing arguments through.*

Arithmetic

```
x=5
(( x += 3 ))              # Modifies x in current shell — x is now 8
echo $((x * 2))           # Arithmetic expansion — prints 16
let y=x*2                 # Alternative
```

> **Note:** *(( )) is a compound command — it runs in the current shell, NOT a subshell. Variables modified inside it persist to the outer script.*

Test Operators

Operator	Meaning
-f file	Regular file exists
-d dir	Directory exists
-e path	Anything exists at path
-z str	String is empty
-n str	String is non-empty
str1 == str2	Strings equal
n1 -eq n2	Numbers equal
n1 -lt n2	n1 less than n2
n1 -gt n2	n1 greater than n2

[[ ]] vs [ ]

`[[ ]]` is a **bash keyword** processed before word splitting. `[ ]` is an external command — subject to word splitting and globbing.

```
# Dangerous with [ ]:
[ -z $VAR ]         # Breaks if VAR has spaces or is unset

# Safe with [[ ]]:
[[ -z $VAR ]]       # Works even without quoting
[[ $str =~ ^[0-9]+$ ]]  # Regex matching — only in [[ ]]
[[ $a == *pattern* ]]   # Glob matching without quoting issues
```

9. Advanced Bash Internals

set Options — Script Safety

Place `set -euo pipefail` at the top of all production scripts:

Option	Flag	Effect
errexit	-e	Exit immediately if any command fails
nounset	-u	Error on undefined variables
pipefail	(none)	Pipeline fails if ANY command in it fails
xtrace	-x	Print each command before executing (debugging)
noexec	-n	Syntax check — read commands but don't run

```
# Without pipefail:
false | true        # Returns 0 — failure SILENTLY IGNORED

# With pipefail:
false | true        # Returns 1 — pipeline correctly reports failure
```

> **Note:** *pipefail returns the exit code of the rightmost command that failed, not necessarily the last command.*

Subshells and Variable Scope — The Pipe Trap

A **subshell** is a child copy of the current shell. Variables set inside a subshell are invisible to the parent:

```
# THE CLASSIC TRAP
cat file.txt | while read line; do
    last=$line        # This runs in a SUBSHELL
done
echo $last            # Empty — variable lost when subshell exited

# FIX 1: Process substitution (loop runs in CURRENT shell)
while read line; do
    last=$line
done < <(cat file.txt)
echo $last            # Works correctly

# FIX 2: Dedicated file descriptor
while read line <&3; do
    last=$line
    # Inner commands can now use stdin (fd 0) freely
done 3< file.txt
```

File Descriptors

```
exec 3< input.txt         # Open fd 3 for reading
exec 4> output.txt        # Open fd 4 for writing
read line <&3             # Read from fd 3
echo "data" >&4           # Write to fd 4
exec 3<&-                 # Close fd 3

# Fix for stdin-consuming inner commands
while read line <&3; do
    ssh server 'command'  # ssh uses real stdin (fd 0), not the file
done 3< hosts.txt
```

Conditional Operators

Operator	Effect
cmd1 && cmd2	Run cmd2 ONLY if cmd1 succeeds (exit 0)
cmd1 || cmd2	Run cmd2 ONLY if cmd1 fails (non-zero exit)
cmd1 ; cmd2	Always run both, sequentially

```
mkdir /tmp/work || exit 1                    # Fail fast
[ -f config.yaml ] && source config.yaml     # Conditional sourcing
cd /deploy && git pull && restart_service    # Chain of dependent steps
```

10. Cron Jobs & Task Scheduling

Cron Syntax

```
┌──────────── minute (0-59)
│ ┌────────── hour (0-23)
│ │ ┌──────── day of month (1-31)
│ │ │ ┌────── month (1-12)
│ │ │ │ ┌──── day of week (0-7, 0 and 7 both = Sunday)
│ │ │ │ │
* * * * * /path/to/command
```

Crontab	Meaning
30 2 * * *	Every day at 2:30 AM
0 * * * *	Every hour at :00
*/5 * * * *	Every 5 minutes
0 9-17 * * 1-5	Every hour 9am–5pm, Monday–Friday
0 0 1 * *	First day of every month at midnight
30 2 * * 7	Every Sunday at 2:30 AM

The Silent Failure Problem

Cron runs with a **stripped-down environment** — no ~/.bashrc, no custom PATH, no user variables. This is the #1 cause of 'works manually, fails in cron.'

```
# What cron sees:
PATH=/usr/bin:/bin

# Your interactive shell might have:
PATH=/usr/local/bin:/usr/bin:/bin:/home/user/.local/bin

# FIX 1: Set PATH in crontab
PATH=/usr/local/bin:/usr/bin:/bin
30 2 * * * /home/user/backup.sh

# FIX 2: Use full absolute paths inside script
/usr/local/bin/python3 script.py

# FIX 3: Capture output
30 2 * * * /home/user/backup.sh >> /var/log/backup.log 2>&1
```

Managing Crontabs

```
crontab -e          # Edit current user's crontab
crontab -l          # List current user's crontab
crontab -r          # Remove current user's crontab
crontab -u john -l  # List john's crontab (root only)
```

11. System Monitoring & Resource Management

Memory

```
free -h                # Human-readable (GB/MB)
free -m                # In megabytes

# Calculate % available memory:
free -m | grep Mem | awk '{print ($4/$2) * 100}'
```

> **Note:** *'available' vs 'free': free is completely unused. available includes memory the kernel can reclaim from cache — this is the realistic number for new programs.*

CPU & Disk

```
top -bn1               # Non-interactive CPU snapshot (useful in scripts)
uptime                 # Load averages for 1, 5, 15 minutes
nproc                  # Number of CPU cores

df -h                  # Disk free — all mounted filesystems
du -sh /var/log        # Disk usage of a directory
du -sh /* | sort -rh   # Find largest top-level directories
```

Log Monitoring

```
tail -f /var/log/syslog           # Follow in real time
tail -n 100 /var/log/auth.log     # Last 100 lines
tail -f -n 20 /var/log/app.log    # Last 20 lines, then follow
journalctl -f                      # Follow systemd journal
journalctl -u nginx                # Logs for specific service
```

12. Networking Basics & SSH

Core Networking Commands

```
ip addr show              # Show all interfaces and IPs
ping -c 4 google.com      # Test ICMP connectivity (4 times)
curl -s -o /dev/null -w "%{http_code}" https://example.com  # HTTP status
ss -tuln                  # Show listening ports (modern netstat)
lsof -i :80               # What process is using port 80?
```

SSH

```
ssh user@hostname                    # Connect
ssh -p 2222 user@hostname            # Non-standard port
ssh -i ~/.ssh/key.pem user@hostname  # Specify private key

# Generate key pair (on local machine)
ssh-keygen -t rsa -b 4096

# Copy public key to server
ssh-copy-id user@hostname

# SCP — secure copy
scp file.txt user@host:/remote/path/    # Local to remote
scp user@host:/remote/file.txt .        # Remote to local
```

> **Note:** *Private key at ~/.ssh/id_rsa must have permissions 600. Public key at ~/.ssh/id_rsa.pub.*

13. Real-World Scripting Patterns

Error Handling & Cleanup with trap

```
#!/bin/bash
set -euo pipefail

cleanup() {
    echo 'Cleaning up...'
    rm -f /tmp/tempfile_$$
}
trap cleanup EXIT          # Runs on exit (success or failure)
trap 'echo Error on line $LINENO' ERR
```

Retry with Exponential Backoff

```
retry() {
    local attempts=$1
    local delay=1
    shift
    for ((i=1; i<=attempts; i++)); do
        "$@" && return 0
        echo "Attempt $i failed, retrying in ${delay}s..."
        sleep $delay
        delay=$((delay * 2))    # Exponential backoff
    done
    return 1
}

retry 5 curl -f https://api.example.com/health
```

Backup Pattern

```
#!/bin/bash
set -euo pipefail

SOURCE="/var/www"
BACKUP_DIR="/backups"
RETENTION=7
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
BACKUP_FILE="${BACKUP_DIR}/backup_${TIMESTAMP}.tar.gz"

mkdir -p "$BACKUP_DIR"
tar -czf "$BACKUP_FILE" "$SOURCE"
sha256sum "$BACKUP_FILE" > "${BACKUP_FILE}.sha256"
sha256sum -c "${BACKUP_FILE}.sha256" --status

# Rotate old backups
find "$BACKUP_DIR" -name "backup_*.tar.gz" -mtime +"$RETENTION" -delete
```

Argument Validation Pattern

```
#!/bin/bash
usage() {
    echo "Usage: $0 <arg1> <arg2>"
    exit 1
}

if [ $# -lt 2 ]; then
    usage
fi
```

```
SOURCE=$1
DESTINATION=$2
```

14. Common Traps & Gotchas

1. Always Quote Variables

```
# BAD: word splitting if VAR has spaces
for file in $FILES; do ...
# GOOD:
for file in "$FILES"; do ...
```

2. Pipe Creates Subshell — Variables Don't Persist

```
cat file | while read line; do
    count=$((count + 1))    # In subshell!
done
echo $count    # 0 — lost
# FIX: use < <(cat file) instead
```

3. find -mtime Sign Confusion

```
find . -mtime -1    # LAST 24h (newer)
find . -mtime +1    # OLDER than 24h
find . -mtime 1     # EXACTLY 1 day
```

4. Exit Code Masking with Pipes

```
set -e
false | true     # Returns 0 — failure HIDDEN
# FIX: set -o pipefail
```

5. >> vs > — Append vs Overwrite

```
echo "data" > file.txt     # OVERWRITES — destroys content
echo "data" >> file.txt    # APPENDS — safe for logs
```

6. Cron PATH vs Shell PATH

```
# ~/.bashrc PATH NOT loaded by cron
# Use absolute paths or set PATH in crontab
PATH=/usr/local/bin:/usr/bin:/bin
```

7. export Does NOT Go Up

```
export VAR=value   # Available to CHILD processes
                   # NOT visible to PARENT
```

8. Always Use "$@" for Argument Passing

```
run_command() { "$@"; }    # Each arg preserved
run_command() { $*; }      # Spaces in args BREAK this
```

9. rm -rf Safety

```
# NEVER:
rm -rf $DIR/     # If DIR is empty becomes: rm -rf /
# ALWAYS validate:
[[ -n "$DIR" && -d "$DIR" ]] && rm -rf "$DIR"
```

10. 2>&1 Order Matters

```
cmd > file 2>&1   # CORRECT: stderr follows stdout to file
cmd 2>&1 > file   # WRONG: stderr goes to terminal (original stdout)
```

15. Quick Reference Cheat Sheet

Redirection

Syntax	Effect
> file	Redirect stdout (overwrite)
>> file	Redirect stdout (append)
< file	Read stdin from file
2> file	Redirect stderr to file
2>&1	Redirect stderr to wherever stdout goes
&> file	Redirect both stdout and stderr
cmd | tee file	Output to screen AND save to file

find Flags

Expression	Meaning
-name '*.log'	Match by name (case-sensitive)
-iname '*.log'	Match by name (case-insensitive)
-type f	Regular files only
-mtime -1	Modified in last 24h
-mtime +7	Modified more than 7 days ago
-mmin -60	Modified in last 60 minutes
-size +100M	Larger than 100 MB
-exec CMD {} \;	Run CMD for each file
-exec CMD {} +	Run CMD once with all files batched
-delete	Delete matched files

chmod Reference

Octal	Symbolic	Common Use
755	rwxr-xr-x	Scripts and directories
644	rw-r--r--	Regular files, configs
600	rw-------	SSH keys, private files
777	rwxrwxrwx	AVOID — world-writable
700	rwx------	Private executables

Cron Reference

Min	Hour	Day	Month	DoW	Meaning
30	2	*	*	*	Daily at 2:30 AM

Min	Hour	Day	Month	DoW	Meaning
0	*	*	*	*	Every hour
*/5	*	*	*	*	Every 5 minutes
0	9	*	*	1-5	9 AM Mon–Fri
0	0	1	*	*	1st of every month

Focus areas: I/O redirection · find flags & signs · bash quoting · pipe subshells · [[ ]] vs [ ] · cron environment